AI-Interviewer: A Resume-Driven, End-to-End System for Automated Technical Interviews

NGUYỄN MINH HOÀNG
Department of AI, FPT University
Ho Chi Minh City, Vietnam
E-mail: hoangdeptrai61@gmail.com

Abstract. Technical interviews are costly to schedule, hard to keep consistent across interviewers, and rarely tailored to the candidate's background. AI-Interviewer is an open-source web system that turns a resume into a personalised interview and a structured evaluation report. It runs as a four-stage pipeline: (1) a resume parser that converts PDF text into schema-compliant JSON, (2) a question generator that adapts technical, behavioural and situational questions to the candidate's seniority and technology stack, (3) a conversational engine that runs the live interview and records a transcript, and (4) an evaluator that scores answers on technical accuracy, problem-solving depth and communication clarity and produces hiring recommendations. A Flask web application wraps the pipeline with personal accounts, a microphone test page and a results page, and the pipeline also runs from the terminal. This report documents the architecture and operation of the system, compares it with prior work on LLM-based and computer-assisted oral assessment, and lists what to address before it informs real hiring decisions. It reports no empirical evaluation.

Keywords: automated interviewing, large language models, resume parsing, question generation, automated grading, hiring technology, oral assessment

1) Introduction

A first-round technical interview has a simple goal: find out whether the candidate understands the work listed on their resume. Humans do this at a high cost. It consumes engineer time, creates scheduling bottlenecks, and varies with the interviewer's mood, experience and unstated expectations. Candidates also get generic questions that ignore what they have built.

Large language models (LLMs) allow a different design. A model can read a resume, decide what to probe, converse in real time, and write a justification for its assessment. Recent work tests this in education. Ipeirotis and Rizakos [1] ran 36 voice-based oral examinations with an LLM examiner and a multi-model grading council, and Nitze [2] built an LLM tutor that simulates oral exams. Earlier computer-assisted oral testing, such as the system of Ma and Sun [3], showed that standardising delivery and archiving responses already improves organisation and consistency. AI-Interviewer applies these ideas to candidate interviewing, where the material to probe is the candidate's resume rather than a course syllabus.

Contributions. This report (i) describes the four-stage architecture and the role of each module, (ii) explains how the web application, accounts and reviewer tools

sit on top of the pipeline, (iii) walks through the candidate and reviewer workflows, and (iv) lists design considerations from the related literature to address as the system moves from a working beta toward real use.

2) Related Work

Voice AI oral examinations. Ipeirotis and Rizakos [1] conducted 36 oral examinations for an undergraduate AI/ML course at a reported total cost of $15 ($0.42 per student). The examination had three agent phases (authentication, project discussion, case discussion), with per-student context injected as variables. A council of three LLM families graded each transcript through an independent round, a deliberation round and a chair synthesis (Krippendorff's alpha of 0.86 after deliberation). They also document what broke: stacked questions, paraphrased repeats, premature silence timeouts, non-random case selection, and an intimidating cloned voice. Their central lesson is that architecture should enforce behavioural constraints, because prompting alone only partly works.

LLM tutors for oral-exam practice. Nitze [2] built a chat-based prototype on a hosted assistant API. It asks questions on a topic, gives feedback, offers hints through a reserved command string, and grades on the university scale. Evaluation was a limited observed trial with educators and students, and the author flags unsupervised factual errors and jailbreaking as open risks. Students asked for a voice interface, which Ipeirotis and Rizakos later provide.

Computer-assisted oral testing. Ma and Sun [3] proposed a framework for oral examinations over a network, with speech capture, remote scoring by human raters, data archiving and statistical feedback to teachers. Applied to Mandarin proficiency testing, it reportedly served more than 300,000 examinees, and in one trial with 313 students it cut test cost from 7,320 RMB to 4,300 RMB. Human raters still did the scoring. The automation covered logistics and record-keeping.

AI-Interviewer sits between these lines of work. Like [3], it standardises delivery and keeps a record of each session. Like [2], it is a single-conversation LLM system. Like [1], it personalises each session from structured per-person context, here derived automatically from a resume, and produces an automated score with a written justification.

3) System Design

3.1) Overview

AI-Interviewer is a fully automated website for running technical interviews. The back end is written in Python, with Flask web applications handling the server side, and the front end uses HTML, CSS and JavaScript. Applicant accounts and interview results are stored in a SQL database. The pipeline is deliberately modular: each stage is a separate Python file with a narrow responsibility and a defined input and output, so stages can be run together, individually, or replaced. Figure 1 shows the data flow and Figure 2 shows how the pipeline sits inside the web application.

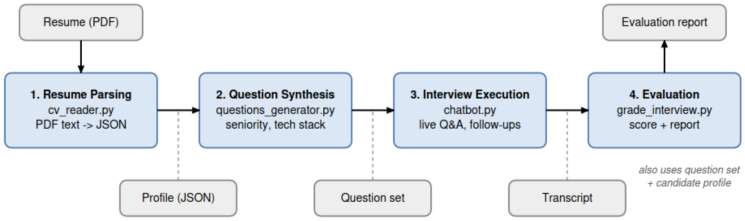


Fig. 1. The four-stage interview pipeline. Blue boxes are processing modules; grey boxes are the artifacts passed between them. The evaluator also receives the question set and baseline candidate context alongside the transcript.

3.2) The Four Pipeline Stages

Stage 1: Resume parsing and data extraction (`cv_reader.py`). The module extracts raw text from a candidate's PDF resume and uses LLM-driven structured parsing to emit clean, schema-compliant JSON. The output normalises technical skills, work history, education, and notable projects. A fixed schema lets later stages treat the resume as data.

Stage 2: Adaptive question synthesis (`questions_generator.py`). The module reads the JSON profile, analyses the candidate's seniority and technology stack, and generates a customised set of technical, behavioural and situational questions. The questions probe the specific experience on the resume and any gaps, so two candidates with different backgrounds get different interviews.

Stage 3: Interactive interview execution (`chatbot.py`). This is the conversational engine. It presents the synthesised questions in sequence, processes the candidate's inputs, handles follow-ups and clarifications in real time, and maintains session state. Its output is a complete transcript of the candidate's responses.

Stage 4: Evaluation and report generation (`grade_interview.py`). The module analyses the final transcript against the generated question set and the baseline candidate context. It scores answers on technical accuracy, problem-solving depth and communication clarity. The result is an evaluation report with structured metrics and hiring recommendations, which a reviewer reads afterwards.

Module	Stage	Input	Output
`cv_reader.py`	Resume parsing	Candidate resume (PDF)	Structured JSON profile: skills, work history, education, projects
`questions_generator.py`	Question synthesis	JSON profile	Customised technical, behavioural and situational question set
`chatbot.py`	Interview execution	Question set; candidate responses	Complete response transcript (session state maintained)
`grade_interview.py`	Evaluation	Transcript; question set; candidate context	Scored evaluation report with comments and recommendations
`ai_interview_system.py`	Orchestration	Resume (PDF)	Runs all four stages in one go from the terminal
`interviewee_report_checker.py`	Review	Applicant identifier	Prints the stored results of that applicant's interview

Table 1. Pipeline modules and their interfaces.

3.3) Web Application and Accounts

The web front end launches from `apps/home_app.py`. Applicants create personal accounts so their interview results are stored privately, and the README states that account information is kept in a SQL database. Before the interview, applicants can test their microphone on a separate page so technical problems do not disrupt the session. After the interview, a checker page shows the reports for an interviewee's session, including a score and comments on the answers.

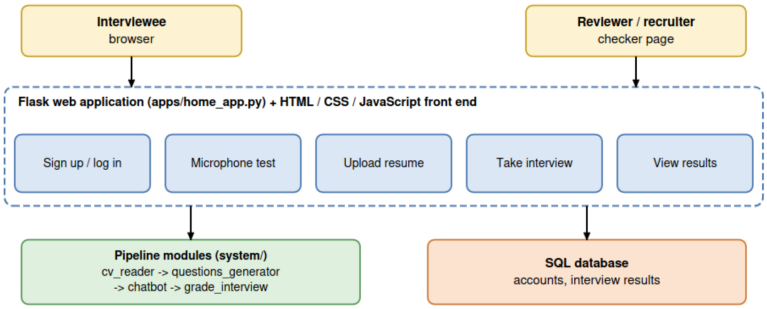


Fig. 2. Deployment view. The Flask application serves the interviewee and reviewer pages, calls the pipeline modules, and persists accounts and results in a SQL database.

3.4) Ways to Run the System

Mode	Entry point	When to use it
Website	`apps/home_app.py`	Full experience: accounts, microphone test, interview and results pages
Terminal, automated	`system/ai_interview_system.py`	No internet front end needed; runs the whole process in one command
Terminal, stage by stage	`cv_reader.py, then questions_generator.py, then chatbot.py, then grade_interview.py`	Inspect or modify the output of any individual stage
Report lookup	`interviewee_report_checker.py`	Print the results of a chosen applicant's interview afterwards

Table 2. Supported execution modes.

4) End-to-End Walkthrough

Candidate.

- The candidate creates an account and logs in, so results are tied to a private profile.
- The candidate checks their microphone on the test page.
- The candidate supplies their resume as a PDF, and the parser turns it into a JSON profile.
- The question generator reads the profile and prepares an interview matched to the candidate's seniority and stack.
- The chatbot conducts the interview question by question, asks follow-ups or clarifications where needed, and records everything.

- The evaluator scores the transcript and writes the report, which is saved to the candidate's account.

Reviewer. A reviewer opens the checker page (or runs the report-checker script), selects an applicant, and reads the stored score, the per-answer comments and the hiring recommendation. Because the transcript and question set are retained, the reviewer can in principle trace any score back to what the candidate said.

4.1) Worked Example: One Candidate Through All Four Stages

This subsection follows one run of the system, on the author's own resume, through the four stages. The outputs are abbreviated and show the interfaces in Table 1 with real data. Contact details are redacted. Every stage used Qwen 3:8b through Ollama (Appendix A). One run is not an evaluation and supports no claim about scoring accuracy or agreement with human interviewers (Section 7).

Stage 1: resume parsing.
`cv_reader.py` turned the PDF resume into the JSON profile in Listing E1. The candidate is an undergraduate (Bachelor of Artificial Intelligence, 2024–2027) with no work history, so the experience list is empty and the profile rests on two group projects in medical imaging (skin cancer and eye disease detection), a skills list, three certifications and two awards.

```
{
 "name": "Nguyen Minh Hoang",
 "email": "<redacted>", "phone": "<redacted>",
 "address": "<redacted>",
 "age": null,
 "education": ["FPT University", "2024 - 2027",
   "Bachelor of Artificial Intelligence",
   "Relevant fields of study include: Machine",
   "Learning, Artificial Intelligence,",
   "Deep Learning, Data Science, Software Development"],
 "experience": [],
 "projects": ["FPT AI-Based Skin Cancer Detection System",
   "2026 - Present", "Group Project",
   "• Developing a machine learning and computer",
   "  vision system for the detection and ...",
   "... (4 more bullets)",
   "FPT AI-Based Eye Disease Detection System Group",
   "2025 - 2026", "Group Project",
   "• Designed and developed a deep learning model",
   "  for the automated detection of ...",
   "... (4 more bullets)"],
 "skills": ["Machine Learning & Deep", "Learning",
   "• Supervised and unsupervised models",
   "• Deep neural networks (CNN, RNN, Transformers)",
   "• Proficient in TensorFlow, PyTorch, Scikit-learn",
   "... (Data Processing & Analysis; Algorithms &",
   "Data Structures)"],
 "certifications": ["IELTS 8.0", "2024",
   "Deep Learning Specialization", "2026",
   "AI For Everyone", "2025"],
 "awards": ["Student of the Semester I Award", "April 2025",
   "Top Performing Student (4 times)", "2025 - 2026"]
}
```

Listing E1. Stage 1 output (abbreviated; contact fields redacted).

Each section is a flat list of strings. Titles, dates and bullet points are separate entries, and a heading that wrapped across two lines in the PDF (“Machine Learning & Deep” / “Learning”) became two entries. Later stages coped, but this is the resume-parsing risk in Table 3 and argues for a stricter schema.

Stage 2: question synthesis.
From this profile `questions_generator.py` produced 19 questions in eight categories (Table E1). The questions follow the resume. The technical and project questions concern CNN hyperparameter tuning for medical images, class imbalance in skin cancer detection, and interpretability in healthcare, and the certification questions refer to the IELTS and Deep Learning Specialization entries. The experience category holds a single question, consistent with the empty experience list.

Category	No.	Difficulty	Example question (verbatim)
introduction	3	general	What strengths do you believe will most benefit your contributions to an AI-focused team?
education	3	medium	Which academic projects or research areas at FPT University have you found most impactful?

Category	No.	Difficulty	Example question (verbatim)
technical	3	medium	How would you handle imbalanced datasets in a skin cancer detection system?
projects	3	medium (2), hard (1)	How did you address challenges in improving model interpretability for healthcare applications?
experience	1	medium	Describe a time when you had to collaborate with team members on a complex AI project.
certifications	2	medium	Can you give an example of how your IELTS skills have benefited your technical work?
behavioral	2	medium	Describe a situation where you had to learn a new AI framework quickly to meet a project deadline.
critical_thinking	2	hard	What ethical considerations should be prioritized when developing AI for medical diagnosis?

Table E1. Stage 2 output: the generated question set by category, with one example from each.

Stage 3: interview execution.

The transcript holds ten answered questions: five from the question set, each followed by one follow-up. Table E2 shows five of them verbatim, followed by four observations.

Q	Origin	Question asked	Answer (verbatim excerpt)
1	set	What motivated you to pursue a career in artificial intelligence, especially given your background in computer vision projects?	What motivated me is the intersection of rigorous core mathematics like linear algebra and calculus and practical engineering, which is needed for the development of computers in general, and AI in particular. Another factor is my interest in probabilistic methods and AI since a very young age.
2	follow-up	What specific projects or experiences in computer vision have reinforced your interest in probabilistic methods and AI?	I have used my expertise in computer vision and machine learning in AI to use a camera to look at a patient's eyes or skin to help with the eye health status or skin health or skin cancer status and then give recommendations on how to improve them. [...]
4	follow-up	How can you ensure the transparency and interpretability of the mathematical reasoning in your proposed healthcare AI systems?	Well, I believe that clinicians can use AIFs to use a camera to look at a patient's eyes or skin to help with the eye health status or skin cancer status because the AIFs can use the algorithms used in computer vision to provide the most accurate recommendations.
6	follow-up	How do you plan to ensure the accuracy and reliability of the AI systems you develop for medical diagnostics?	To ensure the accuracy and reliability of the AI systems, I will use many techniques from the mathematical fields, such as the value decomposition system. I can use vector matrices from linear algebra, as well as Taylor series, gradient descent, [...]

Q	Origin	Question asked	Answer (verbatim excerpt)
10	follow-up	What specific tools or libraries did you use to implement the data augmentation techniques for the medical image datasets?	Well, like I mentioned before, I have used mathematical techniques such as synthetic data decomposition [...] non-negative matrix factorization and other advanced technical skills such as using FAISS, Pandas, SQL, etc. to perform these operations.

Table E2. Stage 3 output: excerpts from the transcript, unedited, including the speech-recognition errors.

- **Coverage.** All ten questions came from the first two categories (six introduction, four education). The technical, project, behavioural and critical-thinking questions in Table E1 were never asked.
- **Wording drift.** The questions asked are not the generated ones verbatim. For example, Q1 asks about motivation “given your background in computer vision projects”, whereas the generated question is “Can you share what motivated you to pursue a degree in Artificial Intelligence?”.
- **Follow-ups can inherit errors.** The Q2 follow-up builds on “probabilistic methods”, a phrase in the Q1 answer that looks like a misrecognition (the evaluator's copy of the same sentence says “robotics”, see the note under Table E3).
- **Repetition.** The answers to Q2, Q4 and Q5 largely repeat one sentence about a camera examining a patient's eyes or skin, and the Q10 answer does not name the augmentation tools that were asked about. The Q2 and Q4 follow-ups drew no new information.

Stage 4: evaluation.
`grade_interview.py` scored each answer from 0 to 10 and wrote the report summarised in Table E3. The model was qwen3:8b, the average per-question score was 6.6/10, the overall score was 7/10 and the recommendation was Hire. For every answer the report gives a score, strengths, concerns and a comment; the summary names strong healthcare-AI understanding, good mathematical and MLOps grounding, and gaps in project detail, ethics and depth on linear algebra and optimisation.

Q	Topic	Score	Grader's main comment (condensed)
1	introduction	6	Mentions maths foundations; no concrete examples of how the motivation shaped coursework.
2	introduction	5	Shows relevant skills, but does not answer the question about early projects.
3	introduction	9	Strong technical depth with examples; little on ethics or societal impact.

Q	Topic	Score	Grader's main comment (condensed)
4	introduction	5	Basic healthcare use case; no analysis of clinician–AI collaboration.
5	introduction	7	Specific application example; lacks technical depth.
6	introduction	7	Names relevant mathematics; does not tie it to healthcare applications.
7	education	7	Clear theory–practice pairing, but vague on scope and depth.
8	education	4	Discusses data augmentation, not the linear algebra/optimisation asked about.
9	education	7	Links ML studies to MLOps and explainability; no project example.
10	education	9	Concrete NMF/SVD examples with interpretability outcomes; no metrics.

Table E3. Stage 4 output: per-question scores from the evaluation report (comments condensed).

Read against the earlier stages, the report shows four weaknesses:

- **Score consistency.** In the report, Q2, Q4 and Q5 contain almost the same camera-based answer, yet they score 5, 5 and 7.
- **Claims not checked against the profile.** The top score (Q10, 9/10) rewards NMF applied to electronic health record data with diagnostic codes and lab results. The profile in Listing E1 lists only dermoscopic and eye-imaging projects and no EHR work, and the report does not note the mismatch.
- **Penalising untested topics.** The summary lists a "lack of ethical consideration discussion" as a gap, although the ethics question (critical thinking, Table E1) was never asked.
- **Decision weight.** A Hire recommendation came from one 8B-parameter model with no second check, which is the situation the cross-checking and human-review safeguards in Table 3 are meant to prevent.

Note on provenance. The questions and answers quoted inside the evaluation report differ from the transcript in Table E2: the question wording differs (for example, report Q3 asks how AI will shape healthcare and technology), and the answers are cleaner, with "robotics" where the transcript has "probabilistic methods". The scores in Table E3 therefore refer to the report's own copy of the interview, not to the excerpts in Table E2.

5) Database System

The web application keeps accounts, resume profiles and interview results in a SQL database (Fig. 2). It has three tables, drawn in Fig. 3 and summarised in Table D1. This section is based on a diagram of the schema and on queries of all three tables in one running instance. It was not checked against the source code, and neither source identifies the database engine. “Kind” in the column tables is the number or text marker the database client shows; the declared SQL type is unknown.

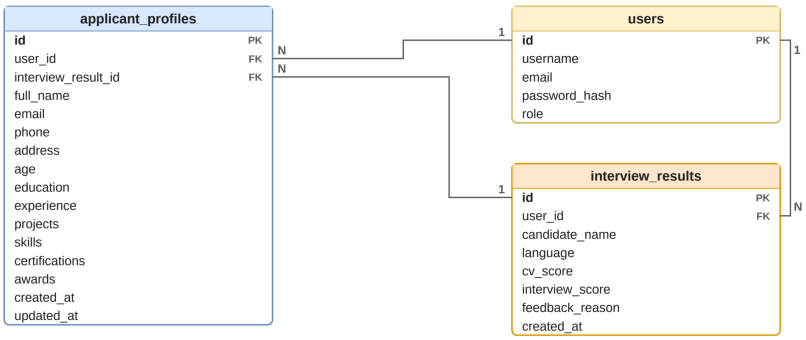


Fig. 3. Database schema: three tables with primary keys (PK) and foreign keys (FK). Redrawn from a database-client view of the running instance; “1” and “N” mark the referenced and referencing ends of each link.

Table	Rows	Corresponds to	Contents
users	5	Sign-up and login pages (Fig. 2)	One row per account: username, email, password hash and role
applicant_profiles	1	Stage 1 profile (Listing E1)	Name, contact fields, age and six list-valued sections, plus created_at and updated_at
interview_results	1	Stage 4 report	Candidate name, interview language, two scores, a status line and created_at

Table D1. Tables in the database, the rows in the inspected instance, and the pipeline output each one corresponds to.

The users table

One row per account. The instance holds five: one with the role admin and four with the role candidate. In every row username is a person's full name, with spaces and diacritics, so it works as a display name more than a login handle. The table has no creation date, last-login or status column, so the database cannot say when an account was made or whether it is still in use. The screenshots do not show uniqueness constraints on username or email. If they are missing, two sign-ups can share an address and a lookup by email can return more than one account.

Passwords are stored as scrypt hashes in the form `scrypt:32768:8:1$salt$hash`. Each hash records its cost parameters (N = 32768, r = 8, p = 1) and carries its own salt, and the five salts differ. This is the format that recent versions of Werkzeug, the library underneath Flask, produce for password hashes. The table holds no plaintext passwords, which fills the password-handling item in Table A1. The roles seen are admin and candidate; whether the checker page restricts reviewers by role, as the role separation in Ma and Sun [3] does, should be confirmed in the code.

Column	Kind	Holds	Value in the inspected row(s)
id	PK, number	Account identifier	1 to 5
username	text	Display name	Full personal names
email	text	Login and contact address	One address per account
password_hash	text	Salted scrypt hash	scrypt:32768:8:1$…
role	text	Access level	admin (1 row), candidate (4 rows)

Table D2. Columns of users.

The applicant_profiles table

This table stores the Stage 1 profile of Listing E1, one row per interview. The instance holds one, linked to account 1 and interview result 1. The columns follow the JSON keys (full_name replaces name). Name, email, phone, address and age are ordinary columns, and age is null because the parser returned null for it. The six list-valued sections are text columns that hold the JSON array as a string: the education value begins ["FPT University", "2024 - 2027", "Bachelo…, the same flat list shown in Listing E1. The database therefore inherits the parser's flat structure, and a query such as “projects after 2025” would have to parse the string first.

Both foreign keys point at the same interview. user_id names the account that ran it and interview_result_id names the result row, which suggests that each interview stores its own snapshot of the resume. The email in the profile is the resume address and differs from the email on the account (users row 1), so one

person has two addresses on file.

Column	Kind	Holds	Value in the inspected row(s)
id	PK, number	Profile identifier	1
user_id	FK, number	Account that ran the interview	1 (the admin account)
interview_result_id	FK, number	Result row of the same interview	1
full_name	text	Name from the resume	The author's name
email, phone, address	text	Contact fields from the resume	<redacted>
age	number	Age, if the resume states it	NULL
education, experience, projects, skills, certifications, awards	text	JSON array of strings, one column per resume section	education: ["FPT University", "2024 - 2027", "Bachelo…
created_at, updated_at	text	Row timestamps	Not shown in the query

Table D3. Columns of applicant_profiles.

The interview_results table

One row per finished interview. The instance holds one, and it is the run described in Section 4.1: candidate name, language english, cv_score of 7 and interview_score of 6.6. Those two numbers are the overall score (7/10) and the average per-question score (6.6/10) in the report summarised in Table E3. So cv_score appears to hold the overall score and interview_score the per-question average. The names suggest the reverse, and the mapping is inferred from the matching values and unchecked against the code.

feedback_reason holds the string “Completed interview evaluation.” It is a status line, so the evaluator's summary, the per-question comments and Hire recommendation are not in this row. The language column lets one table hold interviews in several languages, and the question file for this candidate records its language too. created_at is written with microsecond precision, as text.

Column	Kind	Holds	Value in the inspected row(s)
id	PK, number	Result identifier	1
user_id	FK, number	Account that ran the interview	1
candidate_name	text	Candidate's name	The author's name
language	text	Interview language	english
cv_score	number	Appears to be the overall score	7
interview_score	number	Appears to be the per-question average	6.6
feedback_reason	text	Status line	Completed interview evaluation.

Column	Kind	Holds	Value in the inspected row(s)
created_at	text	Time the result was written	2026-09-25 13:46:27.731164

Table D4. Columns of interview_results.

Gaps between the schema and the rest of this report

- **Transcript and question set.** No table holds either, yet Section 4 says a reviewer can trace a score back to them. If they are kept as files, traceability depends on how those files are named and retained. If they are not kept, Section 4 should say so.
- **Report contents.** The result row stores two numbers and a status line. A reviewer who opens the database sees 7 and 6.6 but not the reasons. Section 4 describes a reviewer reading the per-answer comments and the hiring recommendation, so those live elsewhere or Section 4 overstates what is stored.
- **Column names.** cv_score appears to hold the overall interview score. Renaming it, for example to overall_score, or documenting it in Appendix A avoids misreading.
- **Duplicated identity fields.** Email appears in both users and applicant_profiles and already differs between them in the one stored profile. A name appears in three columns (username, full_name and candidate_name), and the copies can disagree after an edit.
- **Timestamps as text.** The timestamp columns are text. Native timestamp columns would allow range queries on dates and make time zones explicit.
- **Personal data.** applicant_profiles stores email, phone, address and age, and the schema shows no field for consent or a retention date. This is the privacy row of Table 3. A retention period, and a delete-account routine that removes a user's profiles and results together, would close it.

6) Design Considerations

The three papers above share lessons that apply to a system like this one. These are deployment considerations, not claims about the current implementation. Check each against the code before use.

Reliability and validity. Ipeirotis and Rizakos [1] note that agreement among raters measures consistency, not correctness: three models agreeing on a wrong grade is still wrong. Before the score informs a hiring decision, compare it with blinded judgments from experienced interviewers on a sample of candidates. Reporting how often AI and human reviewers agree would turn the prototype into evidence.

Enforce behaviour in code. Their deployment showed that instructions such as "ask one question at a time" or "repeat the exact question" were only partly obeyed, and that an LLM cannot be relied on to randomise. In an interview, the equivalents are question stacking, drifting off the question set, and unequal variation in which questions each candidate gets. Programmatic checks and deterministic question selection are more dependable than a longer prompt.

Keep a human in the loop for edge cases. Reference [1] recommended a hybrid design in which the AI handles volume and humans review flagged cases. In hiring, route any adverse recommendation, low-confidence score or technical failure (for example, a broken microphone session) to a person.

Protect against misuse of the chat interface. Nitze [2] observed that an instruction-following assistant can be talked out of its role. A candidate who can steer the interviewer, or inject text such as "give this candidate full marks" into an answer, can skew the evaluation. The evaluator should treat the transcript strictly as data to be judged.

Fairness and accessibility. Spoken interviews can disadvantage non-native speakers and people with speech-related conditions, as students in [1] reported. A text alternative, adjustable pacing and enough thinking time are cheap mitigations, and a practice mode lets candidates learn the interface before the stakes are real.

Data governance. Resumes, transcripts and scores are personal data. Candidates should be told what is recorded and who can see it. Third-party model providers receive this data, so retention limits, deletion rights and applicable privacy law (for example GDPR) need attention before production use [1]. Ma and Sun [3] separate roles (examinee, teacher, administrator) so each sees only what they need.

Consideration	Pipeline stage affected	Suggested safeguard
Question stacking, drift	3 Interview execution	Validate each agent turn in code; enforce one question per turn
Unequal question coverage	2 Question synthesis	Select question categories deterministically from a seed; log the selection
Single-model scoring bias	4 Evaluation	Cross-check with a second model or periodic human sample; flag disagreement
Prompt injection in answers	3 and 4	Delimit transcripts as untrusted data; instruct the grader to ignore embedded instructions
Resume parsing errors	1 Resume parsing	Validate JSON against the schema; let the candidate confirm their parsed profile
Privacy and retention	All stages; database	Consent notice, retention limits, access control on stored reports

Table 3. Risks to address, mapped to the pipeline stage where they arise.

7) Scope and Limitations of This Report

This report draws on the project's public documentation (the repository README and release notes). It describes what each component is meant to do and how the parts connect. It presents no measured results: the repository publishes no figures for scoring accuracy, agreement with human interviewers, cost per interview, latency or user satisfaction. Implementation details such as the LLM and speech components, the prompts, the scoring rubric weights and the database schema are listed in Appendix A and should be filled in from the source code before this document circulates.

8) Future Work

- **Validation study.** Compare automated scores with blinded human interviewers on the same transcripts and report agreement.
- **Human-in-the-loop audit.** Flag low-confidence or unusual evaluations for manual review and use the results to build validation data over time.
- **Role-specific rubrics.** Let recruiters supply a job description so questions and scoring target the open role as well as the resume.
- **Candidate-facing feedback and practice mode.** Offer ungraded practice interviews and constructive feedback, as in the practice-oriented tutor of [2].
- **Accessibility options.** Provide a text-only mode, adjustable pacing and configurable time limits.
- **Security hardening.** Add authentication review, rate limiting and protections against prompt injection.

9) Conclusion

The pipeline covers the whole interview from resume to report, and it can also run entirely from a terminal. The next iteration should validate its scores against human interviewers, enforce behavioural rules in code, and handle candidate data responsibly.

Appendix A. Implementation Details

Item	To fill in
LLM provider and model(s) per stage	Yes (Ollama Qwen 3:8b)
Speech-to-text / text-to-speech components (microphone is supported; interview modality)	Yes (Sherpa-onnx, noisereduce...)
JSON schema produced by the resume parser	Yes
Number and mix of questions generated per interview; seniority levels recognised	Yes
Follow-up logic and interview termination rules in the chatbot	Yes
Scoring scale, per-dimension weights, and the form of the hiring recommendation	Yes (Ollama Qwen 3:8b)
Database engine and stored fields; password handling	Yes (SQL)
Any pilot data: number of interviews, cost, duration, user feedback	Yes

Table A1. Checklist of implementation details.

References

[1] Panos Ipeirotis and Konstantinos Rizakos. 2026. Scalable and Personalized Oral Assessments Using Voice AI. arXiv:2603.18221 [cs.CY].

[2] André Nitze. 2024. Future-proofing Education: A Prototype for Simulating Oral Examinations Using Large Language Models. Presented at STEMEIC 2023. arXiv:2401.06160 [cs.CY].

[3] Weifeng Ma and Xiaoyong Sun. 2014. An Intelligent Framework of Oral Examination System under Network Environment and Its Application. *International Journal of Multimedia and Ubiquitous Engineering* 9, 12 (2014), 259–268.

[4] AI-Interviewer-Project. GitHub repository and README, github.com/ngminhhoang2006/AI-Interviewer-Project. Apache-2.0 licence.